WSDL to DSA Solution Design

Contents

1 Goal **2**

2 Design summary **2**

3 Problem the feature solves **2**

4 Issue #1824 acceptance alignment **3**

5 Capability design **4**

5.1 Detection and manifest entry contract 4

5.2 Load and generic manifest integration 4

5.3 Contract and operation model ownership 5

5.4 Embedded and referenced schema translation 5

5.5 Namespace-aware QName identity and resolution 5

5.6 Explicit ambiguity and error contract 6

5.7 CLI copy, parity, and downstream compatibility 6

6 Supported contract versus deferred boundaries **6**

6.1 Current supported scope summary 6

6.2 Broader WSDL and SOAP future-work summary 7

7 Requirement and use-case traceability **8**

7.1 Functional requirements 8

7.2 Non-functional requirements 8

7.3 Evidence-layer relation table 9

7.4 Use-case and requirement mapping 10

7.5 Use-case diagram 12

8 Structural interpretation **12**

9 Sequence diagrams **13**

9.1 `spinta copy -o` to WSDL load and DSA output 13

9.2 Namespace-aware schema delegation and parser boundaries 13

This document maintains feature-level design for WSDL-to-DSA issue (#1824) in `atviriduomenys/spinta`. It describes the supported WSDL reader, schema translation, namespace-aware QName, ambiguity, and CLI copy contract as one canonical design. Historical plan, evidence, and archive notebooks remain useful provenance, but they do not define the structure of this document.

1 Goal

Document the generic WSDL-to-DSA feature contract for turning supported WSDL inputs into DSA manifest output through explicit manifest detection, generic loader integration, model generation, embedded and referenced schema handling, namespace-aware resolution, explicit ambiguity behavior, and downstream-compatible CLI output.

2 Design summary

At a high level, the WSDL-to-DSA feature makes eight design commitments:

1. WSDL detection is explicit and predictable for supported local and remote input shapes.
2. WSDL loading fits the repository's generic manifest-loading contract through `read_schema` rather than a bespoke runtime path.
3. Contract-owned raw schema models remain distinct from generated operation request and response models.
4. Embedded and referenced schema content is translated into DSA-visible fields and resources through a WSDL-owned orchestration layer with shared schema-reader delegation.
5. WSDL and embedded-schema references are resolved by expanded QName identity rather than by local-name-only matching.
6. QName resolution uses the active in-scope namespace bindings of the declaring element, supports same-URI different-prefix cases, and keeps same-local-name cross-namespace identities distinct.
7. Genuine ambiguity and duplicate expanded-QName conflicts fail explicitly through a WSDL-owned user-facing error contract instead of being silently guessed, renamed, or overwritten.
8. Supported WSDL output remains aligned with standalone `xsd` and `xsd2` semantics, preserves supported SOAP/resource metadata, and stays loadable by downstream manifest consumers.

3 Problem the feature solves

The WSDL-to-DSA feature fills a specific gap: translating supported WSDL inputs into DSA manifest output while staying consistent with existing `spinta` manifest conventions.

The feature therefore has to solve more than XML parsing alone. It defines:

1. How WSDL inputs are detected and normalized before load begins.
2. How WSDL reading fits the same generic schema-yielding flow used by other manifests.
3. How raw contract models, operation models, and schema-derived fields coexist without collisions.
4. How embedded and referenced schema material is shaped into DSA-compatible output while preserving shared schema semantics.
5. How namespace-aware QName identity, default namespaces, and cross-namespace references behave in both reader and embedded-schema flows.
6. How ambiguity, degraded schema references, and fatal input failures become explicit product behavior through the CLI and manifest loader.

4 Issue #1824 acceptance alignment

The table below maps the acceptance areas from `notebooks/1824.md` onto the maintained requirement and use-case vocabulary so the supported contract can be reviewed from either direction.

Issue #1824 acceptance area	Maintained design interpretation	Requirement mapping	Use-case mapping	Primary live evidence
Generate DSA from local WSDL file	Supported local `.wsdl` and explicit WSDL-path inputs are detected, loaded through `read_schema`, and rendered into loadable DSA output.	FR-1, FR-2, FR-3, FR-8; NFR-1, NFR-2, NFR-8	UC-1, UC-2, UC-3, UC-8, UC-11	`test_wsdl_functional.py`, `test_copy_wsdl.py`
Generate DSA from remote WSDL URL	Supported remote WSDL URLs are detected and loaded without local-path validation, with explicit user-facing transport and authentication failures.	FR-1, FR-2, FR-7, FR-8; NFR-1, NFR-6, NFR-7, NFR-8	UC-1, UC-2, UC-7, UC-9, UC-11	`test_wsdl_functional.py`, `test_copy_wsdl.py`
Support WSDL 1.1 and WSDL 2.0	Reader selection is driven by the root namespace and both supported versions preserve the covered happy-path manifest contract.	FR-1, FR-2, FR-3, FR-8; NFR-1, NFR-2, NFR-8	UC-1, UC-2, UC-3, UC-8, UC-9	`test_wsdl_functional.py`, `test_copy_wsdl.py`
Interpret WSDL structure into DSA	Services, ports or endpoints, bindings, operations, and request or response model ownership are preserved in the generated manifest shape.	FR-3, FR-4, FR-8; NFR-2, NFR-3, NFR-8	UC-3, UC-4, UC-8, UC-9	`test_wsdl_functional.py`, `test_copy_wsdl.py`
Preserve SOAP metadata	Supported SOAP resources retain endpoint address, binding metadata, and optional SOAP action semantics in DSA output.	FR-4, FR-8; NFR-7, NFR-8	UC-4, UC-8, UC-9	`test_wsdl_functional.py`, `test_copy_wsdl.py`
Delegate XSD work to existing parser path	WSDL owns orchestration and degradation policy while shared schema parsing semantics stay aligned with standalone `xsd` and `xsd2`.	FR-4, FR-5, FR-6, FR-8; NFR-3, NFR-4, NFR-5, NFR-8	UC-4, UC-5, UC-6, UC-10	`test_wsdl_functional.py`, `test_wsdl_xsd_parity.py`

Issue #1824 acceptance area	Maintained design interpretation	Requirement mapping	Use-case mapping	Primary live evidence
Implement `read_schema` correctly	WSDL integrates through the generic manifest-loading contract with dataset-first lazy emission and runtime loadability downstream.	FR-2, FR-3; NFR-2, NFR-3	UC-2, UC-3, UC-11	`test_wsdl_functional.py`
Handle errors and edge cases explicitly	Root-document failures are fatal, ambiguity is explicit, and referenced-schema failures degrade with warnings and partial output when the root WSDL succeeds.	FR-2, FR-4, FR-7; NFR-6, NFR-7	UC-7, UC-9, UC-10	`test_wsdl_functional.py`, `test_copy_wsdl.py`
Preserve CLI integration and backward compatibility	`copy` remains the main user entrypoint and generated schema semantics stay aligned with downstream loader expectations and existing XSD behavior.	FR-8; NFR-2, NFR-8	UC-8, UC-9, UC-11	`test_copy_wsdl.py`, `test_wsdl_xsd_parity.py`

5 Capability design

5.1 Detection and manifest entry contract

The feature introduces an explicit WSDL manifest entry contract.

Design decisions:

1. WSDL detection is driven by supported path markers such as explicit `wsdl:` prefixes, supported file naming, and supported remote query markers rather than by treating every XML file as a WSDL candidate.
2. WSDL version choice is made by reading the root document namespace and dispatching only to supported WSDL reader flows.
3. Backend resolution and inspect-path behavior treat WSDL as a recognized manifest family rather than as an incidental XML subtype.

5.2 Load and generic manifest integration

The feature does not introduce a separate WSDL runtime materializer. Instead, it makes WSDL conform to the existing generic manifest-loading model.

Design decisions:

1. `read_schema` is the canonical WSDL reader seam.
2. The reader yields manifest schema dictionaries lazily instead of materializing everything up front.
3. The emitted sequence begins with the dataset, then continues with raw schema models and operation models.
4. Missing local manifest files are fatal during configuration, while supported remote WSDL sources are allowed to bypass local path checks.
5. Malformed root WSDL content is a fatal user-visible error for both local and remote sources.

5.3 Contract and operation model ownership

The feature establishes that WSDL produces two distinct classes of models.

Design decisions:

1. Raw schema models belong to the contract resource, not to a specific operation resource.
2. Operation request and response models remain operation-specific and keep their own generated identities.
3. Shared extraction context carries both operation-link information and raw-schema output so those concerns can be coordinated without name collisions.
4. Operation-model generation continues to work even when there is no usable embedded schema content.

5.4 Embedded and referenced schema translation

The feature treats WSDL translation as more than operation enumeration. It defines how embedded and referenced schema content becomes DSA-visible output.

Design decisions:

1. Embedded XSD scalar types are mapped into DSA datatypes, including the user-visible `anyURI` to `url` override.
2. Nested complex types are flattened into stable field names and source paths instead of being preserved as deep XML-shaped object trees.
3. Referenced local and remote schemas are resolved through a WSDL-owned orchestration seam that delegates schema parsing and composition to the shared XSD path.
4. Reference resolution is recursion-safe.
5. When referenced schemas are missing, malformed, or unreachable, the feature prefers degraded output and recorded diagnostics over aborting the entire manifest generation.
6. Root-document failures remain fatal even though non-root referenced-schema failures are degradable.

5.5 Namespace-aware QName identity and resolution

The feature makes QName identity explicit and uses it across reader and embedded-schema flows.

Design decisions:

1. QName-bearing values are carried as structured identity with raw value, local name, namespace URI, and optional prefix.
2. Expanded QName, not local name alone, is the stable lookup key for covered WSDL and embedded-schema references.
3. `_read_xml()` captures per-element in-scope namespace maps while parsing the root document.
4. QName resolution uses the declaring element's active namespace bindings instead of a flattened document-global prefix table.
5. Same namespace URI with different prefixes resolves to the same supported identity.
6. Same local names across different namespace URIs remain distinct valid identities and must not collapse into one lookup bucket.

7. Covered default-namespace and cross-namespace references are supported when the delegated schema graph resolves them unambiguously.

5.6 Explicit ambiguity and error contract

The feature deliberately chooses explicit failure over silent conflict resolution.

Design decisions:

1. QName-bearing references must resolve to exactly one safe expanded-QName outcome.
2. Duplicate expanded-QName conflicts in scopes that require uniqueness fail explicitly instead of being silently overwritten, renamed, or guessed.
3. Fatal WSDL read, transport, authentication, malformed-input, and namespace-ambiguity failures surface as user-visible non-traceback errors.
4. Referenced-schema failures may emit warnings while still writing manifest output when the root WSDL is readable.

5.7 CLI copy, parity, and downstream compatibility

The feature treats `copy` as the main end-user contract and parity with shared schema readers as an acceptance gate.

Design decisions:

1. `copy` accepts supported local and remote WSDL sources and renders DSA output that remains loadable.
2. Supported WSDL happy paths preserve operation structure, nested fields, scalar mappings, supported SOAP binding metadata, endpoint URLs, and optional SOAP action behavior.
3. WSDL-generated schema output remains aligned with standalone `xsd` and `xsd2` user-visible field semantics for covered shapes.
4. Internal ownership differences are allowed as long as emitted manifest semantics remain aligned.

6 Supported contract versus deferred boundaries

6.1 Current supported scope summary

In user-facing terms, the current supported WSDL-to-DSA contract is this:

1. `copy` accepts supported local and remote WSDL inputs and produces loadable DSA manifest output.
2. The WSDL reader supports the covered WSDL 1.1 and WSDL 2.0 SOAP-compatible happy paths.
3. Manifest output preserves contract-owned raw schema models, operation request and response models, supported SOAP metadata, and downstream loadability.
4. Embedded and referenced schemas are delegated to the shared XSD path while WSDL keeps orchestration, namespace policy, ambiguity handling, and warning-based degradation ownership.
5. Namespace-aware QName identity, same-URI different-prefix equivalence, default-namespace handling, and covered cross-namespace references are part of the supported contract.

6. Root-document failures are fatal and user-visible, while non-root referenced-schema failures may degrade with warnings and partial output.

6.2 Broader WSDL and SOAP future-work summary

The current contract is intentionally narrower than the broader WSDL and SOAP space. The following remain explicit future work rather than implied partial support:

1. Advanced WSDL 2.0 behavior beyond the current SOAP-compatible happy path, including richer message patterns, broader interface composition, and full fault-policy support.
2. Non-SOAP binding support and any policy for treating non-SOAP WSDL documents as part of the supported copy surface.
3. Multipart request and response handling beyond the current covered shapes.
4. Runtime SOAP execution, backend execution flows.
5. Broader detection-policy questions such as plain `.xml` WSDL sniffing outside the supported explicit markers and namespaces.
6. Repository-wide XML or QName refactoring outside the WSDL-owned contract boundary.
7. Review-oriented documentation polish such as additional entity diagrams, relationship views, or presentation-pack material.

The supported feature contract is intentionally narrower than full WSDL coverage.

Supported in this feature:

1. Explicit WSDL manifest detection for supported local and remote path shapes.
2. WSDL reader selection from the document namespace for supported WSDL inputs.
3. Generic manifest loading through `read_schema` with lazy dataset-first emission.
4. Contract-owned raw schema plus operation model generation without name collisions.
5. Embedded schema translation and referenced-schema delegation for supported shapes, including warning-based degradation when non-root referenced schemas fail.
6. Namespace-aware QName identity, same-URI different-prefix equivalence, default-namespace handling, and cross-namespace identity preservation for covered reader and embedded-schema flows.
7. Explicit `AmbiguousWsdlReference` behavior for genuine non-unique QName outcomes or duplicate expanded-QName conflicts in scopes that require uniqueness.
8. CLI copy generation with user-visible fatal errors, warning-based degradation, and loadable DSA output.
9. Manifest-level parity with standalone `xsd` and `xsd2` pipelines for covered schema semantics.

Explicitly deferred or out of scope:

1. Broad plain-XML WSDL sniffing outside the supported markers and namespaces.
2. Runtime SOAP execution or backend execution redesign.
3. Silent renaming, automatic conflict resolution, or other non-explicit ambiguity policy.
4. Full schema-validity enforcement or broader multi-namespace composition policy beyond the covered subset.
5. Advanced WSDL 2.0 behavior outside the supported happy path, including deferred fault and non-SOAP binding policy questions.
6. Repository-wide XML namespace or QName utility extraction outside the WSDL-owned flow.

7 Requirement and use-case traceability

The tables below define one canonical requirement vocabulary for the WSDL-to-DSA feature. Requirement and use-case identifiers are feature-level and remain stable even when implementation history changes.

7.1 Functional requirements

ID	Requirement	Derived from accepted behavior
FR-1	The system must detect supported WSDL manifests from explicit local and remote path markers, normalize supported input forms, and select the appropriate WSDL reader from the document namespace.	Detection and manifest-entry behavior
FR-2	The system must load WSDL manifests through `read_schema` and the generic manifest loader, yielding manifest schema dictionaries lazily in dataset-first order and surfacing fatal root-document failures as user-visible errors.	Generic loading and fatal-failure behavior
FR-3	The system must generate contract-owned raw schema models separately from operation request and response models without name collisions and without suppressing operation generation when embedded schema content is absent.	Raw-schema and operation-model behavior
FR-4	The system must translate embedded and referenced schema content into DSA-visible manifest output, including scalar mapping, nested-field flattening, supported SOAP/resource metadata, shared-schema delegation, and warning-based degradation for non-root referenced-schema failures.	Schema translation and delegation behavior
FR-5	The system must preserve QName identity as expanded QName metadata and use that identity as the lookup basis for covered WSDL and embedded-schema references.	QName identity behavior
FR-6	The system must resolve QName-bearing values against the declaring element's in-scope namespace bindings, support same-URI different-prefix and default-namespace cases, and preserve distinct same-local-name cross-namespace identities.	Namespace-aware resolution behavior
FR-7	The system must raise explicit user-facing ambiguity failures when QName resolution is not unique or when a duplicate expanded-QName conflict appears in a scope that requires uniqueness, and the CLI must render those failures without traceback leakage.	Ambiguity and CLI error-surface behavior
FR-8	The `copy` CLI must translate supported local and remote WSDL inputs into loadable DSA manifest output that preserves supported happy-path structure and remains aligned with standalone `xsd` and `xsd2` semantics for covered schema features.	CLI copy, parity, and downstream compatibility behavior

7.2 Non-functional requirements

ID	Requirement	Derived from accepted behavior
NFR-1	Detection boundaries must be predictable and explicit, avoiding broad plain-XML WSDL sniffing semantics outside the supported markers and namespaces.	Detection behavior
NFR-2	WSDL support must fit existing `spinta` manifest conventions and generic loading contracts instead of introducing a bespoke WSDL-only runtime path.	Detection and load integration behavior
NFR-3	Extraction must remain incremental and reusable, relying on lazy iteration, shared extraction context, and a WSDL-owned orchestration seam rather than duplicated full-document passes.	Load, raw-schema, and delegation behavior
NFR-4	Namespace behavior must be based on namespace URI and expanded QName identity rather than prefix spelling alone.	QName identity and namespace-aware resolution behavior
NFR-5	QName resolution must be scope-aware, using active in-scope namespace bindings, while valid same-local-name cross-namespace identities remain isolated instead of collapsing into local-name-only lookup.	Scope-aware resolution and cross-namespace identity behavior
NFR-6	Ambiguity and failure policy must be explicit and user-visible rather than relying on silent renaming, hidden conflict resolution, or traceback leakage.	Ambiguity and CLI error behavior
NFR-7	Referenced-schema failures must degrade predictably with preserved partial output and recorded diagnostics, while root-document failures remain fatal.	Delegation and warning behavior
NFR-8	Generated manifest output must remain semantically aligned with established `xsd` and `xsd2` behavior and stay loadable by downstream manifest consumers, while broader XML refactors and advanced WSDL policy remain explicit deferred boundaries.	Parity, loadability, and deferred-boundary behavior

7.3 Evidence-layer relation table

Capability slice	Primary live evidence layers	Functional requirements	Non-functional requirements	Use cases populated from this relation
Detection and manifest entry	`test_wsdl_functional.py` for supported path and version behavior, plus `test_copy_wsdl.py` for user-facing invocation paths	FR-1	NFR-1, NFR-2	UC-1, UC-2, UC-8, UC-9
Load and generic manifest integration	`test_wsdl_functional.py` as the primary manifest baseline, with `test_wsdl.py` retained only for secondary seam-oriented checks around orchestration reuse	FR-2	NFR-2, NFR-3, NFR-6	UC-2, UC-11
Contract and operation model ownership	`test_wsdl_functional.py` as the main manifest baseline	FR-3	NFR-2, NFR-3	UC-3, UC-8, UC-9

Capability slice	Primary live evidence layers	Functional requirements	Non-functional requirements	Use cases populated from this relation
Schema translation and delegation	`test_wsdl_functional.py` for supported shaping and degradation behavior, `test_wsdl_xsd_parity.py` for schema-semantic alignment, and `test_copy_wsdl.py` for warning-based partial generation	FR-4	NFR-3, NFR-7, NFR-8	UC-4, UC-10
Namespace-aware QName identity and resolution	`test_wsdl_functional.py` as the main manifest baseline, with `test_wsdl.py` retained only where the supporting seam probes still explain non-user-visible boundary details	FR-5, FR-6	NFR-4, NFR-5	UC-5, UC-6
Explicit ambiguity and CLI error surface	`test_wsdl_functional.py` for manifest-level ambiguity behavior plus `test_copy_wsdl.py` for non-traceback CLI rendering	FR-7	NFR-6, NFR-7	UC-7, UC-8, UC-9
CLI copy, parity, and downstream compatibility	`test_copy_wsdl.py` and `test_wsdl_xsd_parity.py`	FR-8	NFR-7, NFR-8	UC-8, UC-9, UC-10, UC-11

7.4 Use-case and requirement mapping

Use case	Primary actor	Goal	Includes	Extends or is extended by	Requirement mapping
UC-1 Identify and normalize WSDL input	CLI user or manifest author	Provide a supported WSDL path or URL that the system recognizes as WSDL and normalizes correctly.	None	Included by UC-2, UC-8, UC-9	FR-1; NFR-1, NFR-2
UC-2 Load WSDL through the generic manifest pipeline	Generic manifest loader	Read a WSDL manifest through `read_schema` and materialize runtime nodes through the shared loader path.	UC-1, UC-3	None	FR-1, FR-2; NFR-2, NFR-3, NFR-6
UC-3 Generate contract and operation models	Generic manifest loader	Produce dataset, raw schema models, and operation models without collisions and without losing operation coverage.	UC-4, UC-5	Included by UC-2, UC-8, UC-9	FR-2, FR-3; NFR-2, NFR-3

Use case	Primary actor	Goal	Includes	Extends or is extended by	Requirement mapping
UC-4 Translate embedded and referenced schemas	Generic manifest loader	Shape embedded and referenced schema content into DSA-visible fields, resources, diagnostics, and supported SOAP metadata.	None	Extended by UC-10	FR-4, FR-8; NFR-3, NFR-7, NFR-8
UC-5 Resolve namespace-aware WSDL and schema links	Generic manifest loader	Resolve QName-bearing WSDL and embedded-schema references through expanded-QName identity and the declaring element's namespace bindings.	UC-6	Included by UC-3	FR-5, FR-6; NFR-4, NFR-5
UC-6 Preserve cross-namespace identity during translation	Generic manifest loader	Keep same-local-name cross-namespace WSDL and embedded-schema identities distinct while still allowing same-URI different-prefix equivalence.	None	Included by UC-5	FR-5, FR-6; NFR-4, NFR-5
UC-7 Surface ambiguity explicitly	CLI user or manifest loader	Receive a clear ambiguity error when QName resolution is not unique, without silent renaming and without CLI traceback leakage.	None	Extends UC-5	FR-7; NFR-6
UC-8 Copy local WSDL to DSA manifest	CLI user	Translate a local WSDL input into DSA CSV output that preserves the supported contract and remains loadable.	UC-1, UC-3, UC-11	Extended by UC-10	FR-1, FR-3, FR-4, FR-7, FR-8; NFR-6, NFR-7, NFR-8
UC-9 Copy remote WSDL to DSA manifest	CLI user	Translate a remote WSDL input into DSA CSV output with explicit user-facing transport and authentication failures when the source cannot be read.	UC-1, UC-3, UC-11	Extended by UC-10	FR-1, FR-3, FR-4, FR-7, FR-8; NFR-6, NFR-7, NFR-8
UC-10 Continue with warning-based partial generation	CLI user	Receive partial but still usable output when referenced schemas fail after the root WSDL has been read successfully.	None	Extends UC-4, UC-8, UC-9	FR-4, FR-8; NFR-7, NFR-8
UC-11 Validate downstream manifest compatibility	CLI user or downstream loader	Confirm that generated manifest output remains loadable and compatible with the downstream manifest pipeline.	None	Included by UC-8, UC-9	FR-2, FR-8; NFR-8

7.5 Use-case diagram

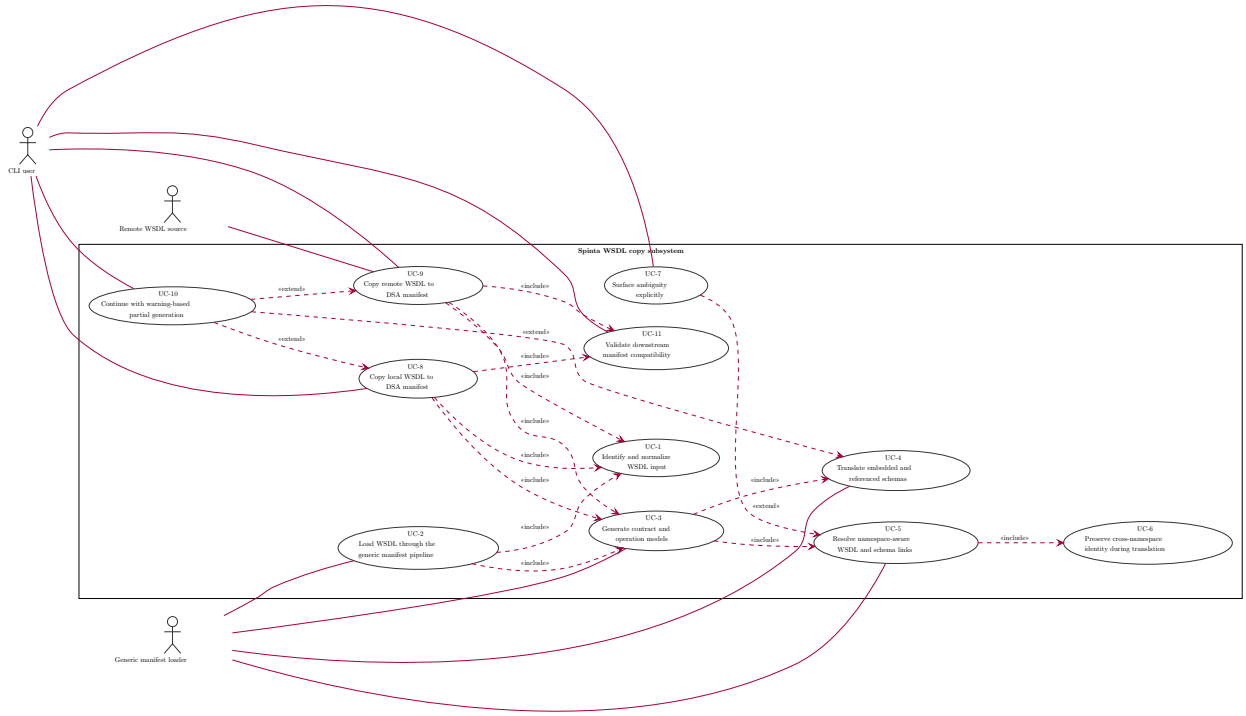

Figure 1: Use-case view for the Spinta WSDL copy subsystem.

8 Structural interpretation

Once the behavior is defined by the live evidence stack, the implementation can be read through a small set of stable public boundaries instead of a helper-by-helper ownership map.

The maintained structural interpretation is:

1. The manifest-entry boundary detects supported WSDL inputs and routes them into the generic manifest command flow.
2. `read_schema` is the canonical WSDL reader seam and the stable handoff into generic manifest loading.
3. QName parsing and namespace resolution form one explicit identity boundary shared by reader and schema-delegation flows.
4. WSDL owns orchestration, ambiguity policy, diagnostics, and degradation behavior around embedded and referenced schemas.
5. Shared `xsd` and `xsd2` parsing remains the schema-semantics boundary that WSDL delegates to rather than replaces.
6. `copy`, tabular rendering, and downstream manifest loading remain the public output and compatibility boundaries.

This structural layer supports the design, but the supported contract still comes from the functional-

first manifest, parity, and CLI evidence layers first.

9 Sequence diagrams

9.1 `spinta copy -o` to WSDL load and DSA output

This sequence starts at the user-visible CLI command and follows the stable public boundaries into WSDL-specific loading. Private helper layout stays intentionally abstracted behind those seams.

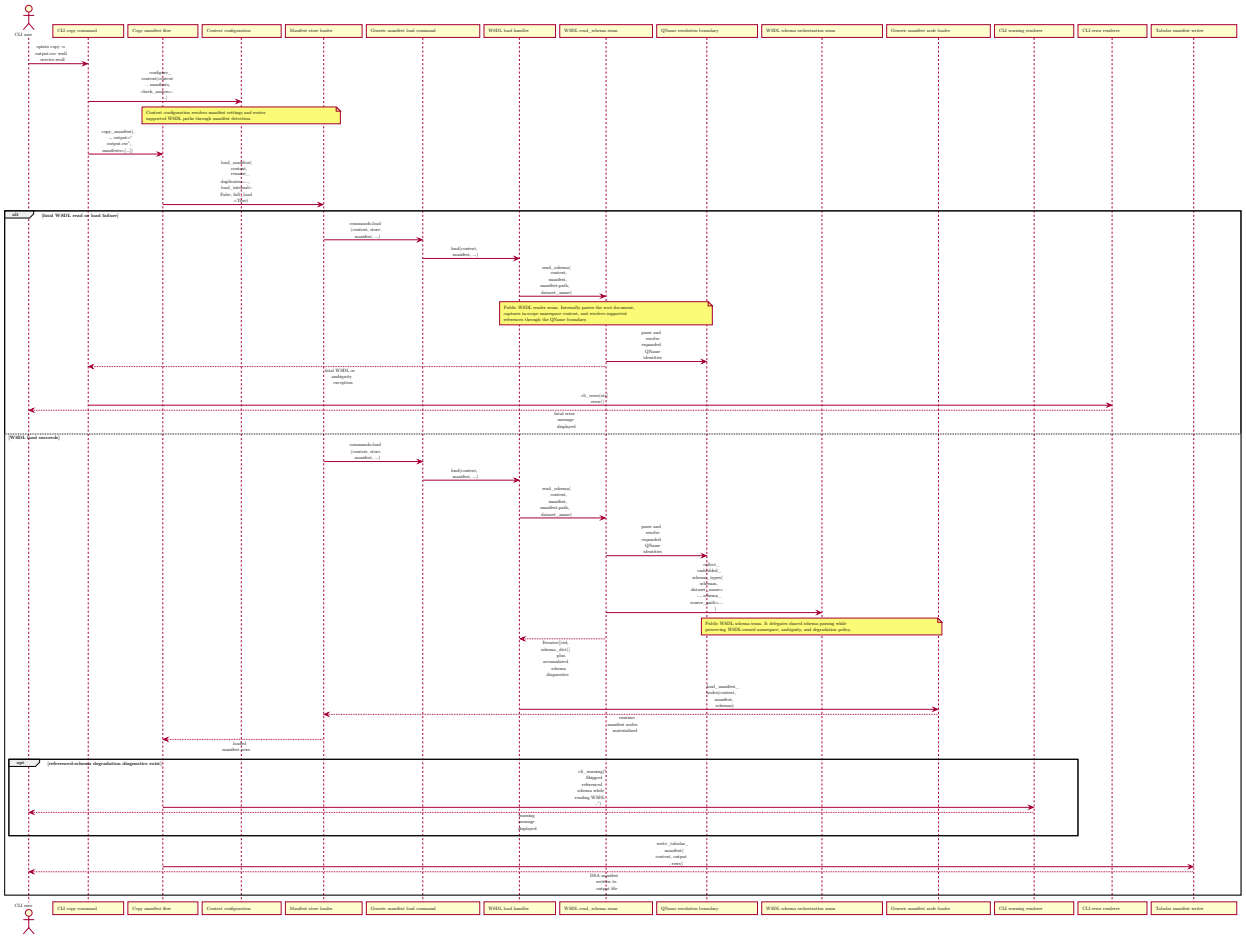

Figure 2: Sequence from `spinta copy -o` through WSDL loading and DSA output generation.

9.2 Namespace-aware schema delegation and parser boundaries

This sequence isolates the schema-delegation boundary. It shows WSDL-owned orchestration crossing into QName resolution, referenced-schema loading, and the shared non-WSDL XSD parser without turning those shared parser semantics into a second WSDL implementation.

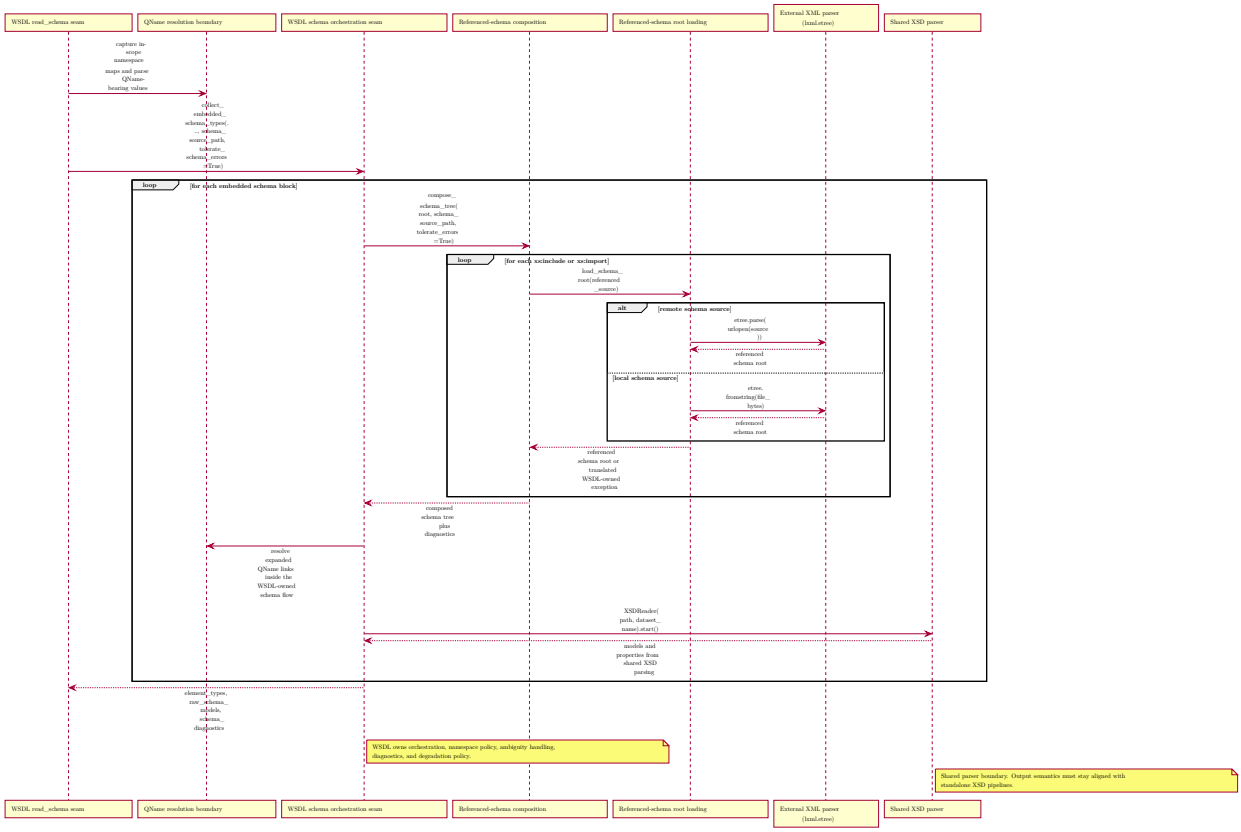

Figure 3: Sequence for QName-aware schema delegation and the shared XSD parser boundary.